Activity 7: Expose a Controller-Based CRUD API

Course: Build ASP.NET Web Apps with Vibe Coding (TGS-2023040477)

Alignment: LO4 · K4/A4

Duration: 65 minutes

Runnable snapshot: src/LocalCRM

Goal

Add a JSON controller surface that reuses the application service without exposing persistence entities.

What you will produce

controller routes, DTOs, 201/200/400/404 behaviour and curl evidence

Safety and data boundary

Use only the supplied fictional records in mock-data/. Do not paste personal, customer, credential, or production data into an AI tool. Review every generated diff before running it.

Vibe script

> Add an ApiController at /api/contacts that supports GET collection, GET item and POST create. Use transport DTOs, IContactStore, CreatedAtAction and CancellationToken. Do not return persistence entities directly. Include 400 and 404 evidence.

Step-by-step

Step 1 — Enable controllers

Register controller services and map controller endpoints.

Create or update Program.cs additions:

```
builder.Services.AddControllers();
// after middleware
app.MapControllers();
```

Step 2 — Create DTOs

Separate client input from response shape.

Create or update LocalCRM/Contracts/ContactDtos.cs:

```
using System.ComponentModel.DataAnnotations;
namespace LocalCRM.Contracts;
public sealed record CreateContactDto([Required, StringLength(80, MinimumLength=2)] string
Name, [Required, EmailAddress] string Email, [Required] string Stage);
public sealed record ContactDto(int Id, string Name, string Email, string Stage,
DateTimeOffset CreatedAt);
```

Step 3 — Extend the service

Add GetAsync(int, CancellationToken) to IContactStore and both implementations.

Step 4 — Create ContactsController

Use [ApiController], [Route], [HttpGet] and [HttpPost]; map entity to DTO explicitly.

Step 5 — Return 201 correctly

Use CreatedAtAction(nameof(Get), new { id }, dto) so the Location header is generated from routing.

Step 6 — Probe list

Start the app and request the collection.

```
curl -i http://localhost:5050/api/contacts
```

Expected evidence: HTTP 200 and a JSON array.

Step 7 — Probe create

Send a valid JSON command.

```
curl -i -X POST http://localhost:5050/api/contacts -H 'Content-Type: application/json' -d
'{"name":"Deepa Nair","email":"deepa@example.test","stage":"Lead"}'
```

Expected evidence: HTTP 201, JSON body and Location header.

Step 8 — Probe failures

Send an invalid DTO and request an absent id.

```
curl -i -X POST http://localhost:5050/api/contacts -H 'Content-Type: application/json' -d
'{"name":"","email":"not-email","stage":"Lead"}'
curl -i http://localhost:5050/api/contacts/9999
```

Expected evidence: HTTP 400 ProblemDetails, then HTTP 404.

Run the supplied snapshot

The Activity folder already contains a complete stage-appropriate ASP.NET Core project and mock data. From this Activity folder:

```
cd src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5107
```

Open http://localhost:5107. For Activities 7–10, also probe http://localhost:5107/api/contacts; for Activities 8–10, probe /openapi/v1.json and /health/ready.

Acceptance checklist

□ The project targets net10.0 and restores without an unreviewed package.

□ The app starts from this Activity folder without relying on another Activity.

□ The supplied mock contacts are fictional and load as described.

□ Positive and negative paths match the expected status or UI state.

□ The diff contains only intended files and no secret.

Troubleshooting

- NETSDK1045: install a .NET 10 SDK or select the installed 10.0 feature band in global.json.
- Address already in use: stop the previous Activity process or use --urls http://localhost:PORT with a free port.
- HTTP 404: compare the exact route, method, route constraint and Activity stage.
- HTTP 400/415: inspect ProblemDetails and confirm JSON shape plus Content-Type: application/json.
- SQLite failure (Activities 8–10): remove only the local training localcrm.db, then restart so the snapshot recreates mock data.

Reflection and evidence record

1. Explain how the evidence demonstrates LO4 and K4/A4.

2. Name one AI suggestion you rejected or changed and why.

3. Identify one negative path you tested and the observable result.

4. State the next improvement you would make without widening the approved scope.

Evidence item	Record the path, command result or observation
Build or test result	
Runtime URL and HTTP status	
Positive-path evidence	
Negative-path evidence	
Reviewed AI change or rejection	
Commit or evidence reference	